

Python – Advanced BL4 Case Studies

1. Student Performance Analysis — List + Tuple

A college stores the following student records:

```
students = [  
    ("Arun", [78, 85, 92, 67]),  
    ("Bala", [45, 67, 55, 49]),  
    ("Charan", [90, 92, 88, 95]),  
    ("Divya", [65, 72, 70, 68]),  
    ("Esha", [88, 76, 91, 84])  
]
```

Analyze the student data and perform the following operations:

1. Calculate the **total and average** marks of every student.
2. Identify the student with the **highest total**.
3. Identify the student with the **lowest average**.
4. Find students whose average is **greater than the class average**.
5. Find students who scored **above 80 in at least three subjects**.
6. Find the **highest mark in each subject**.
7. Identify students who scored **below 50 in any subject**.
8. Sort the students based on their average marks in descending order.
9. Identify the **top two students**.
10. Explain which parts of the structure are mutable and which are immutable.

Expected Output:

Student Totals:

Arun : 322

Bala : 216

Charan : 365

Divya : 275

Esha : 339

Student Averages:

Arun : 80.50

Bala : 54.00

Charan : 91.25

Divya : 68.75

Esha : 84.75

Highest Total: Charan

Lowest Average: Bala

Above Class Average:

Arun, Charan, Esha

Students with 3 or more marks above 80:

Charan

Subject-wise Highest:

Subject 1: 90

Subject 2: 92

Subject 3: 92

Subject 4: 95

Students with a mark below 50:

Bala

Top 2:

Charan

Esha

2. Customer Name Analysis — String + List

An e-commerce company receives customer names in inconsistent formats:

```
names = [  
    " Arun Kumar ",  
    "ARUN KUMAR",  
    " arun kumar",  
    "Bala Krishnan",  
    "BALA KRISHNAN ",  
    "Charan",  
    " charan ",  
    "DIVYA DEVI",  
    "Divya Devi"  
]
```

Analyze the customer data.

Tasks

1. Remove unnecessary leading and trailing spaces.
2. Convert all names to lowercase.
3. Identify duplicate customers.
4. Count the number of unique customers.
5. Find customers whose names contain more than one word.

6. Find the longest customer name.
7. Find customers whose names start with 'a' or 'd'.
8. Count the number of characters in each normalized name.
9. Sort the unique names alphabetically.
10. Determine which customer name occurs most frequently.

Expected Output:

Normalized:

```
['arun kumar', 'arun kumar', 'arun kumar',  
'bala krishnan', 'bala krishnan',  
'charan', 'charan',  
'divya devi', 'divya devi']
```

Unique Customers:

```
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Number of Unique Customers: 4

Duplicate Customers:

arun kumar

bala krishnan

charan

divya devi

Names with Multiple Words:

arun kumar

bala krishnan

divya devi

Longest Name:

bala krishnan

Starts with A or D:

arun kumar

divya devi

3. Product Inventory Analysis — List + Tuple

A supermarket stores product information as:

products = [

 ("Laptop", 5, 55000),

 ("Mouse", 25, 700),

 ("Keyboard", 8, 1200),

 ("Monitor", 12, 15000),

 ("Printer", 4, 12000),

 ("Headset", 15, 2500)

]

Each tuple contains:

(Product Name, Quantity, Price)

Analyze the inventory.

Tasks

1. Calculate the **inventory value** of every product.
2. Calculate the **total inventory value**.
3. Identify products whose quantity is less than 10.

4. Identify products whose price is greater than ₹10,000.
5. Find the most expensive product.
6. Find the product having the highest inventory value.
7. Sort products based on price.
8. Sort products based on quantity.
9. Calculate the total quantity of all products.
10. Identify products contributing more than ₹50,000 to inventory value.

Expected Output:

Inventory Values:

Laptop : 275000

Mouse : 17500

Keyboard : 9600

Monitor : 180000

Printer : 48000

Headset : 37500

Total Inventory Value: 567600

Low Stock:

Laptop

Keyboard

Printer

Price > 10000:

Laptop

Monitor

Printer

Most Expensive:

Laptop

Highest Inventory Value:

Laptop

Total Quantity: 69

4. Text Analysis — String + List

A university receives the following feedback:

```
feedback = """
```

```
Python programming is easy to learn.
```

```
Python programming is powerful.
```

```
Learning Python requires practice.
```

```
Practice makes programming skills better.
```

```
"""
```

Analyze the text.

Tasks

1. Convert the entire text to lowercase.
2. Remove punctuation.
3. Convert the text into a list of words.
4. Count the total number of words.
5. Find the number of unique words.
6. Find the frequency of "python".

7. Find the frequency of "programming".
8. Identify words appearing more than once.
9. Find the longest word.
10. Sort the unique words alphabetically.
11. Find all words containing the letter 'p'.
12. Find the percentage of unique words.

Expected Output:

Total Words: 21

Unique Words: 16

Frequency of python: 3

Frequency of programming: 3

Repeated Words:

python

programming

is

learning

practice

Longest Word:

programming

Words containing 'p':

python

programming

powerful

practice

5. Employee Skill Analysis — Tuple + List + String

An organization stores:

```
employees = [  
    ("E101", "Arun", ["Python", "SQL", "ML"]),  
    ("E102", "Bala", ["Java", "SQL", "HTML"]),  
    ("E103", "Charan", ["Python", "ML", "SQL"]),  
    ("E104", "Divya", ["Python", "Java", "Cloud"]),  
    ("E105", "Esha", ["Python", "ML", "Cloud"])  
]
```

Analyze the employee skill data.

Tasks

1. Find all employees who know Python.
2. Find employees who know both Python and ML.
3. Find employees who know SQL but not Java.
4. Find employees who know either Java or Cloud.
5. Count employees having each skill.
6. Identify the most common skill.
7. Identify employees having at least three skills.
8. Find employees who do not know Python.
9. Sort employees based on the number of skills.
10. Determine which employees are suitable for a **Python + ML project**.

Expected Output:

Python:

Arun, Charan, Divya, Esha

Python + ML:

Arun, Charan, Esha

SQL but not Java:

Arun, Charan

Java or Cloud:

Bala, Divya, Esha

Employees with 3 skills:

Arun, Bala, Charan, Divya, Esha

Without Python:

Bala

Most Common Skill:

Python

Suitable for Python + ML:

Arun, Charan, Esha

6. Transaction Analysis — Tuple + List

A bank stores transactions:

transactions = [

```
("T101", "Deposit", 5000),  
("T102", "Withdraw", 2000),  
("T103", "Deposit", 8000),  
("T104", "Withdraw", 1500),  
("T105", "Deposit", 3000),  
("T106", "Withdraw", 7000)  
]
```

Analyze the transaction history.

Tasks

1. Calculate total deposits.
2. Calculate total withdrawals.
3. Calculate the net balance.
4. Find the largest deposit.
5. Find the largest withdrawal.
6. Identify all transactions greater than ₹4,000.
7. Count deposits and withdrawals.
8. Find the average transaction amount.
9. Sort transactions based on amount.
10. Identify whether total withdrawals exceed 50% of total deposits.

Expected Output:

Total Deposit: 16000

Total Withdrawal: 10500

Net Balance: 5500

Largest Deposit: 8000

Largest Withdrawal: 7000

Transactions > 4000:

T101

T103

T106

Deposits: 3

Withdrawals: 3

Average Transaction: 4416.67

Withdrawals > 50% of Deposits:

Yes

7. Password Security Analysis — String + List

A system receives:

```
passwords = [  
    "Python@123",  
    "python123",  
    "PYTHON@123",  
    "Py@45",  
    "Python123",  
    "Java#4567",  
    "Admin@2026"  
]
```

A valid password must have:

- At least 8 characters
- Uppercase
- Lowercase
- Digit
- Special character

Analyze:

1. Check every password against all five conditions.
2. Identify valid passwords.
3. Identify invalid passwords.
4. For each invalid password, identify the failed conditions.
5. Count valid and invalid passwords.
6. Find the longest password.
7. Find passwords containing "python" irrespective of case.
8. Sort passwords based on length.
9. Determine the percentage of valid passwords.

Expected Output:

Valid:

Python@123

Java#4567

Admin@2026

Invalid:

python123 → No uppercase, No special character

PYTHON@123 → No lowercase

Py@45 → Less than 8 characters

Python123 → No special character

Valid Count: 3

Invalid Count: 4

Longest Password:

Python@123

Python-related passwords:

Python@123

python123

PYTHON@123

Python123

Valid Percentage: 42.86%

8. Code Analysis and Debugging — List + Tuple + String

Consider the following program:

```
records = [  
    ("Arun ", "Python"),  
    ("ARUN", "Python"),  
    ("Bala ", "Java"),  
    ("bala", "Java"),  
    ("Charan", "Python")  
]
```

```
names = []
```

```
for record in records:
```

```
    name = record[0].strip().lower()
```

```
    if name not in names:
```

```
        names.append(name)
```

```
print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find **unique employees and their skills**.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

```
['arun', 'bala', 'charan']
```

9. Word Frequency and Data Analysis — String + List + Tuple

Consider:

```
text = "data science uses data analysis and data visualization"
```

Analyze the string using Python.

Tasks

1. Convert the sentence into a list of words.
2. Count the total words.
3. Find unique words.
4. Find repeated words.
5. Count the frequency of every word.
6. Identify the most frequently occurring word.
7. Create tuples of the form:

(word, frequency)

8. Sort the tuples based on frequency.
9. Find words having frequency greater than 1.
10. Identify the longest word.
11. Find words beginning with 'd'.
12. Calculate the percentage of repeated words.

Expected Output:

Total Words: 9

Unique Words:

['data', 'science', 'uses', 'analysis', 'and', 'visualization']

Frequency:

data → 3

science → 1

uses → 1

analysis → 1

and → 1

visualization → 1

Most Frequent:

data

Repeated Words:

data

Longest Word:

visualization

10. Integrated Student Data Analysis — String + List + Tuple

A college maintains:

```
students = [  
    (" A001 ", "ARUN KUMAR", [78, 85, 92]),  
    ("A002", "Bala", [45, 67, 52]),  
    (" a003", "arun kumar", [88, 91, 95]),  
    ("A004", "CHARAN", [72, 76, 81]),  
    ("A005", "Divya ", [55, 48, 62])  
]
```

Each record contains:

(Student ID, Student Name, Marks)

Analyze the complete dataset:

1. Normalize all student IDs using strip() and upper().
2. Normalize all student names using strip() and lower().
3. Calculate total and average marks.

4. Identify students who passed all subjects.
5. Identify students who failed at least one subject.
6. Find the highest total.
7. Find the lowest total.
8. Identify students with average above 75.
9. Detect duplicate student names after normalization.
10. Sort students based on average marks.
11. Find the top two students.
12. Find the subject-wise highest marks.
13. Find the subject-wise average marks.
14. Count the number of students who scored above 80 in at least two subjects.
15. Identify the student who has the highest mark in Subject 3.
16. Explain which data structure is appropriate for **student records**, **marks**, and **individual student identity**, and justify your answer.

Expected Output:

Normalized Records:

A001 arun kumar [78, 85, 92]

A002 bala [45, 67, 52]

A003 arun kumar [88, 91, 95]

A004 charan [72, 76, 81]

A005 divya [55, 48, 62]

Totals:

arun kumar → 255

bala → 164

arun kumar → 274

charan → 229

divya → 165

Averages:

arun kumar → 85.00

bala → 54.67

arun kumar → 91.33

charan → 76.33

divya → 55.00

Passed All Subjects:

arun kumar

arun kumar

charan

Failed At Least One:

bala

divya

Highest Total:

arun kumar → 274

Lowest Total:

bala → 164

Average > 75:

arun kumar

arun kumar

charan

Duplicate Name:

arun kumar

Top 2:

arun kumar → 91.33

arun kumar → 85.00

Subject-wise Highest:

Subject 1 → 88

Subject 2 → 91

Subject 3 → 95

Subject-wise Average:

Subject 1 → 67.60

Subject 2 → 73.40

Subject 3 → 78.40